🏏 CRICBUZZ LIVESTATS:

REAL-TIME CRICKET INSIGHTS & SQL-BASED ANALYTICS

INTRODUCTION

Cricbuzz LiveStats is a high-performance sports analytics platform that harmonizes real-time data acquisition with deep-dive historical analysis. Developed using **Python**, **Streamlit**, and **SQL**, the platform serves as a unified ecosystem where users can track live match momentum, explore detailed player profiles, and execute complex analytical queries. It transforms raw API data into strategic intelligence.

PROBLEM Statement

The sports industry frequently struggles with fragmented data sources. While fans have access to live scores, they often lack a tool that integrates **live performance** with **historical career data** and **custom database management**.

- **Data Latency:** Standard platforms often have significant delays in score reporting.
- **Lack of Customization:** Users cannot run their own SQL-based analytics on live data.
- **Static Information:** Historical stats are often trapped in separate databases from live match events.

OBJECTIVE

- **Real-Time Data Orchestration:** Implement a robust pipeline to fetch live series and player stats using the Cricbuzz REST API.
- **Relational Persistence:** Design a SQL schema to store and manage players, matches, and career stats.
- **Analytical Deep-Dives:** Implement 25+ SQL queries to solve complex business questions like "Clutch Performance" and "Venue Consistency."
- **Data Governance:** Build a full CRUD interface for administrative data management.

🚀 FEATURES

- 📊 **Live Cricket Updates** – Fetches real-time match details, including scores, status, and venues.
- 📝 **Scorecards & Player Insights** – View batting, bowling, and player statistics at a glance.
- 🎯 **Interactive Streamlit Dashboard** – A clean, responsive UI with filtering options.
- 🗄️ **Database Support** – A MySQL backend for data persistence.
- 🔍 **SQL Query Playground** – Write and execute custom SQL queries directly inside app.
- 🛠️ **CRUD Operations** – Add, update, delete, and view cricket data in real time.

🚀 TECH STACK

- **Python** → Data fetching & processing.
- **Streamlit** → Interactive dashboard.
- **MySQL** → Database storage & SQL analytics.
- **REST API (Cricbuzz via RapidAPI)** → Live Cricbuzz cricket data.
- **pandas** → Data manipulation.
- **requests** → API integration.

APPROACH:

```
cricbuzz_livestats/
├── app.py                  # Main entry point for the Streamlit app
├── requirements.txt        # Required Python packages
├── README.md               # Project overview and setup instructions
│
├── pages/                  # Contains individual Streamlit pages
│   ├── home.py             # Overview and About the Project page
│   ├── live_matches.py     # Displays live match data from Cricbuzz API
│   ├── top_stats.py        # Shows top batting/bowling stats
│   ├── sql_queries.py      # SQL query interface and analytics
│   └── crud_operations.py  # Perform CRUD on player stats
│
└── utils/                  # Utility files
    └── db_connection.py    # SQL database connection logic
```

DATABASE SCHEMA:

The project integrates with the Cricbuzz API (via RapidAPI) and also uses JSON datasets exported from the API. These datasets are stored in MySQL (cricket_db).

Tables Created:

- teams → Team information (team_id, team_name, short_name).
- players → Player details (id, name, batting style, bowling style, country, team_id).
- matches → Match details (id, series, format, teams, venue, status).
- scores → Score details (runs, wickets, overs, innings).
- venues → Venue details (name, location, capacity, profile).

- series → Series information (series_id, name, type, start_date, end_date).
- player_stats → Player performance statistics (matches, innings, runs, average, type).

METHODOLOGY

1. Data Collection

- **Live match data** fetched from Cricbuzz API (/matches/v1/live).
- **Player stats** from TopStats API (/stats/v1/topstats).
- **Venues** from (/venues/v1).
- **Series** from (/series/v1).
- Stored exported **JSON files** locally (all_team_players.json, all_venues.json, recent_matches.json, player_stats.json).

2. Database Setup

- Created relational tables in **MySQL**.
- Linked players → teams, matches → series, matches → venues.
- Inserted JSON/API data into respective tables using utility functions in utils.py.

3. Application Development

Built using:

- **Streamlit** → For interactive dashboards and query execution.
- **MySQL** → To store and query structured cricket data.
- **Pandas** → For query results and transformations.
- **JSON** → To store raw player details, matches, and statistical data fetched from API.
- **Cricbuzz API** → For retrieving live matches, player stats, venues, and other cricket-related data.

📊 DASHBOARD VISUALIZATION

1. 📈 KPI Overview

- **Live Database Metrics:** Real-time counters showing the total volume of matches, teams, and players currently tracked.
- **Analytical Snapshots:** Instant visibility into key figures like "Series Active" and "Total Venues" stored in the MySQL backend.
- **Purpose:** Provides an immediate technical snapshot of the platform’s data scale and current reach.

2. 🔍 Browse & Filter Section

- **Multi-Dimensional Filtering:** Sift through data based on **Match Format** (Test, ODI, T20), specific **Series**, or participating **Teams**.
- **Dynamic Data Tables:** An interactive table view powered by **Pandas**, displaying real-time team names, match status, and scores.
- **Purpose:** Streamlines navigation, allowing users to quickly pinpoint specific results within a large dataset of historical and live matches.

3. 🗄️ SQL Query Insights

Empowers users to execute complex analytical deep-dives through direct database interaction. Sample implemented queries include:

- **Regional Analytics:** Identifying all players representing a specific country (e.g., India).
- **Team Dominance:** Aggregating total match wins per team across all formats.
- **Performance Leaders:** Extracting top run-scorers and leading wicket-takers from the player_stats table.
- **Venue Infrastructure:** Ranking stadiums by highest spectator capacity.
- **Purpose:** Provides stakeholders with advanced analytical power to transform raw database records into strategic sports intelligence.

4. ⚙️ CRUD Operations

A dedicated administrative interface for direct database governance:

- **Create:** seamlessly insert new player profiles, teams, and match schedules into the relational schema.
- **Read:** Instant retrieval of current player roles and match data via the interactive UI.
- **Update:** Modify existing records, such as player roles or team affiliations, with real-time database sync.
- **Delete:** Securely remove outdated or incorrect records to maintain data integrity.
- **Purpose:** Ensures the platform remains a "living" ecosystem, manageable directly by users without requiring raw SQL command access.

🎯 KEY FEATURES WALKTHROUGH

📊 Live Matches Dashboard

- Auto-Refresh: The dashboard updates every 30 seconds for live scores.
- Filters: You can filter matches by format, status, and venue.
- Match Details: Click on a match to view ball-by-ball information.
- Visuals: See real-time match statistics and data visualizations.

📈 Top Stats & Analytics

- Batting Leaders: View leaderboards for runs, averages, strike rates, and boundaries.
- Bowling Leaders: See top bowlers by wickets, economy rates, and maidens.
- Team Trends: Compare teams across different cricket formats.
- Data Management: Quickly refresh, clear, or regenerate data.

🗄 SQL Query Playground

- Pre-Built Queries: Get quick insights from a list of pre-built queries for common stats.
- Custom Query Builder: Write and execute your own SQL queries.
- Schema Explorer: An interactive browser to view your database structure.

⚙ CRUD Operations

- Player Management: Add, update, and remove player information.
- Match Management: Manage match schedules and results.
- Performance Data: Insert or clean up batting and bowling statistics.

🔍 ANALYTICAL INSIGHTS

- **International Dominance:** Top-tier international players consistently anchor the leaderboards for both runs and wickets across all formats.
- **Infrastructure Hotspots:** High-capacity venues are geographically concentrated in India and Australia, reflecting major spectator interest.
- **Offensive Correlation:** Data shows a direct link between deep batting lineups and higher overall match win counts.
- **Domestic Impact:** Domestic T20 and List-A series contribute significantly to the cumulative career metrics stored in the player_stats table.
- **Format Versatility:** Multi-format players tend to have higher ICC rankings, showing a correlation between consistency and high-frequency play.
- **Venue Advantage:** Analysis of historical match data suggests that specific teams maintain a statistically significant "home-ground" win rate.
- **Strike Rate Trends:** Modern T20 players prioritize high strike rates over batting averages, a trend clearly visible in the recent API-fetched datasets.
- **Bowling Efficiency:** Spinners show better economy rates in subcontinental venues, while pacers dominate the wicket counts in pace-friendly conditions like Perth or London.
- **Data Integrity:** Relational linking between series and matches reveals that series duration and format directly impact player injury and rotation frequencies.

CONCLUSION

The **Cricbuzz LiveStats System** serves as a robust demonstration of how sports data can be seamlessly collected, stored, and analyzed in a unified environment. By bridging the gap between live API streams and relational database management, it provides a comprehensive toolkit for modern cricket analytics.

The platform successfully enables:

- **Real-time tracking** of live cricket matches with automated score updates.
- **Data-driven insights** into player performance, team trends, and venue history.
- **Interactive SQL exploration**, allowing users to run complex custom queries for deep-dive analysis.
- **Multi-source integration**, consolidating disparate API endpoints and JSON datasets into a structured MySQL backend.
- **Streamlined Data Governance** through a dedicated CRUD interface for efficient database administration.

This project highlights the potential for building scalable, high-performance sports analytics platforms using a modern Python and SQL tech stack.